

Especificación Técnica

Procedimientos Almacenados y Endpoints

Sistema de Gestión de Asistencias — Universidad Católica de Oriente



SQL Server — Transact-SQL



Spring Boot — Clean Architecture



RBAC — 5 Perfiles

Versión 1.0 — Septiembre 2026

Operaciones atómicas orientadas a dominio · CQRS · Evitando CRUDs genéricos

Tabla de Contenidos

Convenciones y Principios de Diseño

PARTE 1 — Especificación de Transacciones por Módulo

Módulo 1: Control de Asistencia (Core Transaccional)

Módulo 2: Gestión de Sesiones y Calendario

Módulo 3: Matrículas e Inscripciones Académicas

Módulo 4: Solicitudes de Revisión de Asistencia (Reclamos)

Módulo 5: Gestión de Identidad Institucional y Roles

Módulo 6: Cierre Masivo de Periodo Académico

Módulo 7: Gestión de Grupo del Docente

PARTE 2 — Matriz Técnica de Mapeo

Control de Asistencia

Gestión de Sesiones

Matrículas e Inscripciones

Solicitudes de Revisión (Reclamos)

Gestión de Identidad y Roles RBAC

Gestión Académica (Grupos, Asignaturas, Planes)

Cierre Masivo de Periodo

Operaciones Simples sin SP (Backend Directo)

Consultas y Lecturas (Vistas uv_*)

Catálogo de Procedimientos Internos Reutilizables

Convenciones y Principios de Diseño

⚠ PRINCIPIO FUNDAMENTAL

Este documento define operaciones **atómicas orientadas a intención de dominio**, no CRUDs genéricos. Cada procedimiento encapsula una transacción de negocio completa con sus validaciones, bloqueos de concurrencia y auditoría.

SP Orquestador (usp_*)

Procedimiento expuesto al Backend vía `SimpleJdbcCall` o `JdbcTemplate.queryForMap`. Retorna siempre el bloque unificado `(idCorrelacion, mensajeUsuarioResultado, mensajeTecnicoResultado, estadoResultado)`.

SP Interno (usp_*_interno)

Procedimiento invocado exclusivamente por orquestadores. Recibe/retorna parámetros `OUTPUT`. **Nunca** invocado directamente desde Spring Boot.

Operación Simple

Mutación de 1 entidad / 1 campo sin lógica multientidad → se ejecuta con `JdbcTemplate` parametrizado en el `*RepositorySqlServerAdapter` del Backend, **sin SP**.

Lectura / Consulta

Se ejecuta contra vistas (`uv_*`) desde el Backend con `NamedParameterJdbcTemplate` y paginación nativa `OFFSET/FETCH`. **Sin SP**.

Estándar Transaccional

Todo SP orquestador debe incluir: `SET XACT_ABORT ON; SET NOCOUNT ON; BEGIN TRY ... BEGIN TRANSACTION ... COMMIT ... END TRY BEGIN CATCH ... ROLLBACK ... END CATCH`.

PARTE 1: Especificación de Transacciones por Módulo

Módulo 1: Control de Asistencia (Core Transaccional)

Tablas Implicadas

TABLA	COLUMNAS CLAVE	FK PRINCIPALES
Asistencia	id, estudianteGrupo, sesion	FK → EstudianteGrupo, FK → Sesion
DetalleAsistencia	id, codigo, asistencia, asistio, razonCausa, fechaHoraInicio, fechaHoraFin	FK → Asistencia, FK → RazonCausa
EstudianteGrupo	id, estado, estudiante, grupo	FK → EstadoEstudianteGrupo, FK → Estudiante, FK → Grupo
Sesion	id, nombre, numero, codigo, numeroSemana, grupo, fechaHoraInicio, fechaHoraFin	FK → Grupo
RazonCausa	id, nombre, codigo	— (paramétrica)

1.1 Registrar Asistencia Masiva por Planilla

ATOMICIDAD: N ESTUDIANTES EN 1 TX VALIDACIÓN POR ESTUDIANTE

Intención: El docente toma asistencia de toda la lista de estudiantes de una sesión en una sola operación.

Lógica de validación:

1. Validar que la sesión exista y no esté cerrada (`usp_validar_sesion_exista_por_id_interno`).
2. Para cada estudiante del lote, validar que pertenezca activamente al grupo de la sesión (`usp_validar_estudiante_pertenece_a_grupo_de_sesion_interno`).
3. Resolver dinámicamente el `RazonCausa.id` a partir del código enviado (ej. 'AN', 'SJC').
4. Insertar o actualizar (MERGE) en `Asistencia` + `DetalleAsistencia` .

INTERNOS INVOCADOS: `usp_validar_id_correlacion_esta_presente_interno` , `usp_validar_sesion_exista_por_id_interno` ,
`usp_validar_estudiante_pertenece_a_grupo_de_sesion_interno` , `usp_sincronizar_asistencia_estudiante_interno`

1.2 Auto-Registro de Asistencia por Token/QR (Estudiante)

ATOMICIDAD: TOKEN + MATRÍCULA + REGISTRO

Intención: El estudiante escanea un código QR o ingresa un PIN de 6 dígitos para marcar su propia asistencia en una sesión abierta.

Lógica de validación:

1. Validar que el token/PIN coincida con el generado para la sesión y no haya expirado (TTL 60s).
2. Validar que la sesión esté dentro de la ventana temporal (inicio – tolerancia ↔ fin).
3. Validar matrícula activa del estudiante en el grupo de la sesión. Si no lo está, **rechazar** (no autovivificar).
4. Validar que no exista ya un registro previo de asistencia (idempotencia).
5. Insertar en `Asistencia` + `DetalleAsistencia` con `RazonCausa = 'AN'`.

INTERNOS INVOCADOS: `usp_validar_sesion_abierta_con_ventana_temporal_interno`, `usp_validar_estudiante_pertenece_a_grupo_de_sesion_interno`,
`usp_sincronizar_asistencia_estudiante_interno`

Módulo 2: Gestión de Sesiones y Calendario

Tablas Implicadas

TABLA	COLUMNAS CLAVE	FK PRINCIPALES
Sesion	id, nombre, numero, codigo, numeroSemana, grupo, fechaHoraInicio, fechaHoraFin	FK → Grupo
Horario	id, grupo, dia, horaInicio, horaFin	FK → Grupo, FK → Dia
Grupo	id, asignatura, periodoAcademico, codigo, nombre, cantidadEstudiantes, docente	FK → Asignatura, FK → PeriodoAcademico, FK → Docente
PeriodoAcademico	id, institucion, nombre, codigo, fechaInicio, fechaFin, anio	FK → Institucion
Dia	id, nombre, codigo	— (paramétrica)

2.1 Crear Sesión Ordinaria/Extraordinaria (con Correlativo Atómico)

BLOQUEO: UPDLOCK, HOLDLOCK

Intención: El docente crea una nueva sesión con número/código secuencial único.

1. Validar correlación.
2. Validar que el grupo pertenezca al docente titular (`usp_validar_grupo_exista_para_docente_interno`).
3. Calcular `MAX(numero) + 1` con bloqueo pesimista.
4. Generar código `SES-XX` .
5. Insertar en `dbo.Sesion` .

2.2 Cerrar Sesión y Congelar Planilla

ATOMICIDAD: CIERRE + AUSENCIAS PENDIENTES

Intención: El docente cierra oficialmente una sesión. Estudiantes sin registro son marcados como ausentes automáticamente.

1. Validar existencia de la sesión y que no esté ya cerrada.
2. Validar titularidad del docente.
3. Insertar registros de ausencia (`asistio = 0` , `RazonCausa = 'SJC'`) para estudiantes activos sin registro.
4. Marcar la sesión como cerrada.

2.3 Cancelar Sesión con Motivo

Intención: El docente cancela una sesión programada registrando el motivo. Las asistencias parciales se anulan.

1. Validar existencia de la sesión y titularidad del docente.
2. Validar que la sesión no esté cerrada con asistencia completa.
3. Anular registros existentes de `DetalleAsistencia`.
4. Marcar la sesión como cancelada con motivo.

2.4 Generar Sesiones Masivas del Semestre

ATOMICIDAD: TODAS LAS SESIONES DEL SEMESTRE EN 1 TX

Intención: Generar automáticamente todas las sesiones para las semanas del periodo académico según los horarios configurados.

1. Validar existencia del grupo (`usp_validar_grupo_exista_por_id_interno`).
2. Validar existencia de franjas horarias (`usp_validar_horarios_grupo_interno`).
3. Validar coherencia de fechas del periodo (`usp_validar_fechas_periodo_academico_interno`).
4. Iterar cada día del calendario cruzando con `Horario`.
5. Insertar en `dbo.Sesion` evitando duplicación por fecha+hora.

Módulo 3: Matrículas e Inscripciones Académicas

Tablas Implicadas

TABLA	COLUMNAS CLAVE	FK PRINCIPALES
EstudianteGrupo	id, estado, estudiante, grupo	FK → EstadoEstudianteGrupo, FK → Estudiante, FK → Grupo
EstudiantePrograma	id, estudiante, programa	FK → Estudiante, FK → Programa
Grupo	id, cantidadEstudiantes, docente, ...	FK → Asignatura, FK → PeriodoAcademico, FK → Docente
EstadoEstudianteGrupo	id, nombre, codigo	— (paramétrica: A, F, CVP, CI)

3.1 Matricular Estudiante en Grupo (Control de Cupo y Cruce Horario)

BLOQUEO: UPDLOCK SOBRE GRUPO **5 VALIDACIONES ENCADENADAS**

Intención: Inscribir a un estudiante garantizando cupo disponible y sin cruce de horarios.

1. Validar existencia y estado activo del estudiante.
2. Validar existencia y habilitación del grupo.
3. Validar unicidad de la matrícula (estudiante no repetido en grupo).
4. Validar cruce de horarios con otros grupos del estudiante.
5. Validar cupo disponible (`cuposDisponibles > 0`).
6. Insertar en `EstudianteGrupo` con estado `'A'` (Activo).

3.2 Retirar Estudiante de Grupo (Cancelación Voluntaria)

Intención: Retirar a un estudiante de un grupo, liberando cupo.

1. Validar que exista la relación activa en `EstudianteGrupo` .
2. Cambiar estado a `'CVP'` (no eliminar físicamente).
3. Actualizar `Grupo.cantidadEstudiantesCancelaronVoluntadPropia` .

3.3 Asignar o Reasignar Docente a Grupo

Intención: Asignar un nuevo docente titular validando disponibilidad horaria.

1. Validar existencia y estado activo del docente.
2. Validar que el grupo existe.
3. Validar cruce de horarios del docente.
4. Actualizar `Grupo.docente` .

3.4 Resolver Solicitud de Matrícula (Coordinador)

ATOMICIDAD: RESOLUCIÓN + MATRÍCULA CONDICIONAL

Intención: El coordinador aprueba o rechaza una solicitud de inscripción. Si aprueba, ejecuta automáticamente la matrícula.

1. Validar existencia de la solicitud en estado pendiente.
2. Validar ámbito RBAC del coordinador sobre el programa.
3. Si acción = `'APROBADA'` : ejecutar `usp_registrar_estudiante_en_grupo_interno` .
4. Si acción = `'RECHAZADA'` : solo actualizar estado.

Módulo 4: Solicitudes de Revisión de Asistencia (Reclamos)

Tablas Implicadas

TABLA	COLUMNAS CLAVE	FK PRINCIPALES
SolicitudRevisionAsistencia	id, nombre, asistencia, fecha, estado, justificacionSolicitud, justificacionRespuesta	FK → Asistencia, FK → Estado
Asistencia	id, estudianteGrupo, sesion	FK → EstudianteGrupo, FK → Sesion
DetalleAsistencia	id, asistencia, asistio, razonCausa	FK → Asistencia, FK → RazonCausa
Estado	id, nombre, codigo	— (paramétrica: A = Aceptada, R = Rechazada)

4.1 Radicar Solicitud de Revisión (Estudiante)

ATOMICIDAD: CREAR ASISTENCIA DE RESPALDO + SOLICITUD

Intención: El estudiante solicita revisión de su asistencia adjuntando justificación y soporte documental.

1. Validar existencia del estudiante y de la sesión.
2. Validar pertenencia al grupo de la sesión.
3. Validar plazo reglamentario (configurable: `ASISTENCIA.DIAS_LIMITE_JUSTIFICACION = 5`).
4. Crear `Asistencia` si no existe. Insertar `SolicitudRevisionAsistencia` con estado `'PEND'`.

4.2 Resolver Solicitud de Revisión (Docente/Coordinador)

ATOMICIDAD: RESOLUCIÓN + MUTACIÓN DETALLEASISTENCIA

Intención: El docente aprueba o rechaza una justificación de inasistencia.

1. Validar que la solicitud exista en estado pendiente.
2. Validar que el docente sea titular del grupo (ámbito RBAC).
3. Si `'APROBADA'`: `UPDATE DetalleAsistencia SET asistio = 1, razonCausa = <ID 'EX'>`.
4. Si `'RECHAZADA'`: solo actualizar estado + respuesta.

Módulo 5: Gestión de Identidad Institucional y Roles

Tablas Implicadas

TABLA	FK PRINCIPALES
Usuario	FK → Tipoidentificacion
Decano , Coordinador , Docente , Estudiante	FK → Usuario
Facultad	FK → Institucion, FK → Decano
Programa	FK → Facultad, FK → TipoPrograma, FK → Coordinador

5.1 Designar Decano y Adscribirlo a Facultad

3 TABLAS: USUARIO + DECANO + FACULTAD

Intención: El administrador crea un nuevo decano y lo asigna como titular de una facultad.

1. Resolver la facultad destino por ID o nombre.
2. Buscar/crear Usuario vía `usp_sincronizar_usuario_interno`.
3. Crear registro Decano si no existe.
4. `UPDATE Facultad SET decano = @idDecano`.

5.2 Designar Coordinador y Adscribirlo a Programa

3 TABLAS: USUARIO + COORDINADOR + PROGRAMA

Intención: El decano crea un coordinador y lo asigna a un programa de su facultad.

1. Validar que el programa pertenezca a la facultad del decano (RBAC).
2. Buscar/crear Usuario vía `usp_sincronizar_usuario_interno`.
3. Crear registro Coordinador si no existe.
4. `UPDATE Programa SET coordinador = @idCoordinador`.

Módulo 6: Cierre Masivo de Periodo Académico

6.1 Ejecutar Cierre Masivo de Periodo

TRANSACCIÓN MASIVA: N GRUPOS × M ESTUDIANTES

Intención: Al finalizar el semestre, procesar todos los grupos del periodo: marcar 'CI' (Cancelado por Inasistencia) a quienes superan el umbral de faltas, 'F' (Finalizado) al resto, y actualizar contadores en Grupo.

1. Resolver periodo académico por código o nombre.
2. Calcular inasistencias acumuladas por estudiante/grupo (umbral: $\geq 20\%$).
3. Actualizar estados masivamente en EstudianteGrupo.
4. Actualizar contadores en Grupo.
5. Insertar evento en AuditoriaEvento.

Módulo 7: Gestión de Grupo del Docente

7.1 Crear Grupo Académico (con Validación de Unicidad de Código)

1. Validar existencia de la asignatura.
2. Resolver periodo académico (si viene nulo, usar el más reciente).
3. Validar existencia y estado activo del docente.
4. Validar unicidad de código de grupo dentro de asignatura/periodo.
5. Insertar en dbo.Grupo con contadores en 0.

PARTE 2: Matriz Técnica de Mapeo

Control de Asistencia

TABLA	TRANSACCIÓN / ACCIÓN DE NEGOCIO	PROCEDIMIENTO ALMACENADO (BD)	MÉTODO DEL CONTROLADOR (BACKEND)	ROL / PERMISO	DESCRIPCIÓN DEL PROCEDIMIENTO
Asistencia , DetalleAsistencia	Registrar asistencia masiva por planilla	usp_registrar_asistencias_sesion	POST /api/v1/asistencias/lote	DOCENTE	Recibe JSON con lista de estudiantes y estado. Valida sesión abierta y pertenencia de cada estudiante. Itera y sincroniza cada asistencia en TX atómica.
Asistencia , DetalleAsistencia	Registrar asistencia individual	usp_registrar_asistencia_estudiante	POST /api/v1/asistencias	DOCENTE	Registra asistencia de un único estudiante. Valida matrícula activa y existencia de sesión. Resuelve RazonCausa por código.
Asistencia , DetalleAsistencia	Auto-registro por QR/PIN	usp_registrar_asistencia_estudiante_autonomo	POST /api/v1/estudiante/asistencia-qr	ESTUDIANTE	Valida token/PIN efímero (TTL 60s). Verifica ventana temporal. Valida matrícula activa sin autovivificación.
Sesion	Generar token/PIN efímero	— (Backend Java)	GET /api/v1/sesiones/{id}/qr-token	DOCENTE	Genera UUID + PIN 6 dígitos con expiración 60s en memoria. Sin SP.

Gestión de Sesiones

TABLA	TRANSACCIÓN / ACCIÓN DE NEGOCIO	PROCEDIMIENTO ALMACENADO (BD)	MÉTODO DEL CONTROLADOR (BACKEND)	ROL / PERMISO	DESCRIPCIÓN DEL PROCEDIMIENTO
Sesion	Crear sesión ordinaria o extraordinaria	usp_crear_sesion	POST /api/v1/sesiones	DOCENTE	Valida titularidad del grupo. Calcula correlativo MAX(numero)+1 con UPDLOCK. Genera código SES-XX.
Sesion , Asistencia , DetalleAsistencia	Cerrar sesión y congelar planilla	usp_cerrar_sesion	POST /api/v1/sesiones/cierres	DOCENTE	Valida existencia y titularidad. Marca ausentes a estudiantes sin registro. Marca sesión cerrada.

TABLA	TRANSACCIÓN / ACCIÓN DE NEGOCIO	PROCEDIMIENTO ALMACENADO (BD)	MÉTODO DEL CONTROLADOR (BACKEND)	ROL / PERMISO	DESCRIPCIÓN DEL PROCEDIMIENTO
Sesion	Cancelar sesión con motivo	usp_cancelar_sesion_con_motivo	PATCH /api/v1/docente/sesiones/{id}/cancelar	DOCENTE	Marca sesión cancelada con motivo. Anula DetalleAsistencia existentes.
Sesion	Generar sesiones masivas del semestre	usp_generar_sesiones_grupo	POST /api/v1/grupos	DOCENTE, COORDINADOR	Genera todas las sesiones del periodo cruzando calendario con franjas horarias. TX completa.
Sesion	Actualizar datos operativos de sesión	usp_actualizar_sesion	PUT /api/v1/sesiones/{id}	DOCENTE	Actualiza nombre, descripción, fechas de sesión no cerrada.

Matrículas e Inscripciones

TABLA	TRANSACCIÓN / ACCIÓN DE NEGOCIO	PROCEDIMIENTO ALMACENADO (BD)	MÉTODO DEL CONTROLADOR (BACKEND)	ROL / PERMISO	DESCRIPCIÓN DEL PROCEDIMIENTO
EstudianteGrupo , Grupo	Matricular estudiante en grupo	usp_registrar_estudiante_en_grupo_interno	POST /api/v1/coordinador/grupos/{id}/estudiantes	COORDINADOR, DOCENTE	Valida existencia estudiante, grupo habilitado, unicidad matrícula, cruce horario, cupo disponible. Inserta con estado 'A'.
EstudianteGrupo , Grupo	Retirar estudiante de grupo	usp_retirar_estudiante_de_grupo	DELETE /api/v1/coordinador/grupos/{id}/estudiantes/{estId}	COORDINADOR, DOCENTE	Cambia estado a 'CVP'. Actualiza contadores de Grupo. No elimina físicamente.
Grupo	Asignar/reasignar docente a grupo	usp_registrar_docente_en_grupo_interno	POST /api/v1/docentes/asignaciones/grupo	COORDINADOR	Valida docente activo, existencia grupo, cruce horario. Actualiza Grupo.docente.
SolicitudMatricula , EstudianteGrupo	Resolver solicitud de matrícula	usp_resolver_solicitud_matricula	PATCH /api/v1/coordinador/solicitudes-matricula/{id}	COORDINADOR	Si APROBADA: ejecuta matrícula con validaciones completas. Si RECHAZADA: solo actualiza estado.

Solicitudes de Revisión (Reclamos)

TABLA	TRANSACCIÓN / ACCIÓN DE NEGOCIO	PROCEDIMIENTO ALMACENADO (BD)	MÉTODO DEL CONTROLADOR (BACKEND)	ROL / PERMISO	DESCRIPCIÓN DEL PROCEDIMIENTO
SolicitudRevisionAsistencia , Asistencia	Radicar solicitud de revisión	usp_radicar_solicitud_revision_asistencia	POST /api/v1/estudiante/reclamos	ESTUDIANTE	Valida estudiante, sesión, pertenencia, plazo 5 días. Crea Asistencia de respaldo si no existe. Inserta con estado 'PEND'.
SolicitudRevisionAsistencia , DetalleAsistencia	Resolver solicitud (aprobar/rechazar)	usp_resolver_solicitud_revision_asistencia	PATCH /api/v1/docente/reclamos/{id}	DOCENTE	Valida solicitud y ámbito RBAC. Si APROBADA: muta DetalleAsistencia.asistio = 1. TX atómica.

Gestión de Identidad y Roles RBAC

TABLA	TRANSACCIÓN / ACCIÓN DE NEGOCIO	PROCEDIMIENTO ALMACENADO (BD)	MÉTODO DEL CONTROLADOR (BACKEND)	ROL / PERMISO	DESCRIPCIÓN DEL PROCEDIMIENTO
Usuario , Decano , Facultad	Designar decano → facultad	usp_crear_decano	POST /api/v1/admin/decanos	ADMINISTRADOR	Busca/crea Usuario vía usp_sincronizar_usuario_interno. Crea Decano. Actualiza Facultad.decano. TX atómica.
Usuario , Coordinador , Programa	Designar coordinador → programa	usp_crear_coordinador	POST /api/v1/decano/coordinadores	DECANO	Busca/crea Usuario. Crea Coordinador. Valida programa. Actualiza Programa.coordinador. TX atómica.
Usuario	Sincronizar usuario institucional	usp_sincronizar_usuario_interno	— (interno)	— (interno)	Valida tipo ID, unicidad correo/documento. Inserta o actualiza Usuario.
Usuario	Activar/Desactivar usuario	— (Backend directo)	PATCH /api/v1/ ... /toggle	ADMIN, DECANO, COORD.	UPDATE dbo.Usuario SET estado = IIF(estado=1,0,1). Sin SP.

Gestión Académica (Grupos, Asignaturas, Planes)

TABLA	TRANSACCIÓN / ACCIÓN DE NEGOCIO	PROCEDIMIENTO ALMACENADO (BD)	MÉTODO DEL CONTROLADOR (BACKEND)	ROL / PERMISO	DESCRIPCIÓN DEL PROCEDIMIENTO
Grupo	Crear grupo académico	usp_crear_grupo	POST /api/v1/grupos	DOCENTE, COORD.	Valida asignatura, resuelve periodo, valida docente activo, unicidad de código. Inserta Grupo con contadores=0.
Grupo	Actualizar datos del grupo	usp_actualizar_grupo	PUT /api/v1/grupos/{id}	DOCENTE, COORD.	Valida existencia, docente si se modifica, unicidad código. Actualización granular.
Asignatura	Crear asignatura en plan	usp_crear_asignatura	POST /api/v1/coordinador/ ... /asignaturas	COORDINADOR	Crea asignatura vinculada a SemestrePlanEstudio. Resuelve Area y Componente.
Asignatura	Actualizar asignatura	usp_actualizar_asignatura	PUT /api/v1/coordinador/ ... /asignaturas/{id}	COORDINADOR	Actualiza código, nombre, créditos, área y componente.
Asignatura	Alternar estado activo/inactivo	usp_toggle_estado_asignatura	PATCH /api/v1/coordinador/asignaturas/{id}/estado	COORDINADOR	Valida existencia. Alterna estado entre 1 y 0 atómicamente.
Asignatura	Eliminar asignatura del plan	usp_eliminar_asignatura	DELETE /api/v1/coordinador/ ... /asignaturas/{id}	COORDINADOR	Valida que no existan grupos activos vinculados antes de eliminar.

Cierre Masivo de Periodo

TABLA	TRANSACCIÓN / ACCIÓN DE NEGOCIO	PROCEDIMIENTO ALMACENADO (BD)	MÉTODO DEL CONTROLADOR (BACKEND)	ROL / PERMISO	DESCRIPCIÓN DEL PROCEDIMIENTO
EstudianteGrupo, Grupo, DetalleAsistencia, AuditoriaEvento	Ejecutar cierre masivo de periodo académico	usp_ejecutar_cierre_masivo_periodo	POST /api/v1/admin/cierre-masivo	ADMINISTRADOR	Resuelve periodo. Calcula inasistencias acumuladas. Marca 'CI' a quienes superen umbral (≥ 3 faltas / $\geq 20\%$). Marca 'F' al resto. Actualiza contadores. Registra auditoría. TX completa.

Operaciones Simples sin SP (Backend Directo)

🔦 NOTA DE DISEÑO

Estas operaciones se ejecutan con `JdbcTemplate` parametrizado en los `*RepositorySqlServerAdapter` del Backend, **sin procedimiento almacenado**, porque son mutaciones de 1 tabla / 1-2 campos sin lógica multientidad.

TABLA	TRANSACCIÓN / ACCIÓN DE NEGOCIO	SP (BD)	MÉTODO DEL CONTROLADOR (BACKEND)	ROL / PERMISO	DESCRIPCIÓN
Facultad	Crear facultad	— (directo)	POST /api/v1/admin/facultades	ADMIN	INSERT con validación de unicidad de nombre.
Facultad	Editar facultad	— (directo)	PUT /api/v1/admin/facultades/{id}	ADMIN	UPDATE nombre/atributos.
Facultad	Activar/Desactivar facultad	— (directo)	PATCH /api/v1/admin/facultades/{id}/estado	ADMIN	Toggle de estado.
Institucion	Crear institución	— (directo)	POST /api/v1/admin/instituciones	ADMIN	Inserción parametrizada.
Institucion	Editar institución	— (directo)	PUT /api/v1/admin/instituciones/{id}	ADMIN	Actualización parametrizada.
Institucion	Activar/Desactivar institución	— (directo)	PATCH /api/v1/admin/instituciones/{id}/toggle-estado	ADMIN	Toggle de estado.
PlanEstudio	Crear plan de estudio	— (directo)	POST /api/v1/coordinador/planes-estudio	COORD.	Inserción con validación de programa.
PlanEstudio	Editar plan	— (directo)	PUT /api/v1/coordinador/planes-estudio/{id}	COORD.	Actualización de INP y estado.
PlanEstudio	Toggle estado plan	— (directo)	PATCH /api/v1/coordinador/ ... /toggle-estado	COORD.	Toggle del campo estado.
SemestrePlanEstudio	Agregar semestre a plan	— (directo)	POST /api/v1/coordinador/ ... /semestres	COORD.	Inserción vinculando con tabla Semestre.
SemestrePlanEstudio	Quitar semestre de plan	— (directo)	DELETE /api/v1/coordinador/ ... /semestres/{num}	COORD.	Eliminación con validación de asignaturas vinculadas.
PeriodoAcademico	Crear periodo académico	— (directo)	POST /api/v1/coordinador/periodos-academicos	COORD., ADMIN	Validación de no solapamiento de fechas.

TABLA	TRANSACCIÓN / ACCIÓN DE NEGOCIO	SP (BD)	MÉTODO DEL CONTROLADOR (BACKEND)	ROL / PERMISO	DESCRIPCIÓN
PeriodoAcademico	Editar periodo	— (directo)	PUT /api/v1/coordinador/periodos-academicos/{id}	COORD., ADMIN	Actualización de nombre, fechas.
Area	Crear/Editar/Toggle área	— (directo)	POST/PUT/PATCH /api/v1/admin/areas	ADMIN	CRUD simple sobre tabla paramétrica.
Horario	Configurar franja horaria	— (directo)	POST /api/v1/grupos	DOCENTE, COORD.	Inserción directa en dbo.Horario.

Consultas y Lecturas (Sin SP — Vistas `uv_*`)

PRINCIPIO CQRS

Todas las lecturas se ejecutan directamente sobre las **43 vistas relacionales** (`uv_*`) desde los adaptadores secundarios de Spring Boot con `NamedParameterJdbcTemplate` y paginación nativa `OFFSET / FETCH` . No requieren procedimientos almacenados.

VISTA	CONSULTA / ACCIÓN	MÉTODO DEL CONTROLADOR (BACKEND)	ROL / PERMISO
<code>uv_grupo</code> , <code>uv_estadistica_grupo</code>	Listar grupos (con filtro por rol)	<code>GET /api/v1/grupos</code>	DOCENTE, COORD., DECANO, ADMIN
<code>uv_sesion</code>	Listar sesiones de un grupo	<code>GET /api/v1/sesiones/grupo/{id}</code>	DOCENTE
<code>uv_estudiante_grupo</code>	Listar estudiantes de un grupo	<code>GET /api/v1/grupos/{id}/estudiantes</code>	DOCENTE, COORD.
<code>uv_asistencia</code> , <code>uv_detalle_asistencia</code>	Consultar asistencias por grupo/sesión	<code>GET /api/v1/grupos/{id}/asistencias</code>	DOCENTE, COORD.
<code>uv_estudiante</code>	Listar/buscar estudiantes del programa	<code>GET /api/v1/coordinador/estudiantes</code>	ESTUDIANTE, COORD.
<code>uv_docente</code>	Listar/buscar docentes	<code>GET /api/v1/coordinador/docentes</code>	COORD., DECANO, ADMIN
<code>uv_coordinador</code>	Listar coordinadores de la facultad	<code>GET /api/v1/decano/coordinadores</code>	DECANO
<code>uv_decano</code>	Listar decanos institucionales	<code>GET /api/v1/admin/decanos</code>	ADMIN
<code>uv_facultad</code>	Listar facultades	<code>GET /api/v1/admin/facultades</code>	ADMIN, DECANO
<code>uv_programa</code>	Listar programas	<code>GET /api/v1/decano/facultad</code>	DECANO, COORD.
<code>uv_plan_estudio</code>	Listar planes de estudio	<code>GET /api/v1/coordinador/planes-estudio</code>	COORD.
<code>uv_asignatura</code>	Listar asignaturas	<code>GET /api/v1/coordinador/asignaturas</code>	COORD.
<code>uv_periodo_academico</code>	Listar periodos académicos	<code>GET /api/v1/coordinador/periodos-academicos</code>	COORD., ADMIN
<code>uv_horario_docente</code>	Consultar horario del docente	<code>GET /api/v1/docente/horarios</code>	DOCENTE
<code>uv_horario_estudiante</code>	Consultar horario del estudiante	<code>GET /api/v1/estudiante/horarios</code>	ESTUDIANTE
<code>uv_solicitud_revision_asistencia</code>	Listar reclamos	<code>GET /api/v1/estudiante/reclamos</code> , <code>GET /api/v1/docente/reclamos</code>	ESTUDIANTE, DOCENTE
<code>uv_tipo_identificacion</code>	Listar tipos de identificación	<code>GET /api/v1/tipos-identificacion</code>	TODOS
<code>uv_usuario_perfil</code>	Consultar perfil del usuario	<code>GET /api/v1/usuarios/perfil</code>	TODOS (autenticado)

Catálogo de Procedimientos Internos Reutilizables (_interno)

PROCEDIMIENTO INTERNO	TIPO	PROPÓSITO	INVOCADO POR
usp_validar_id_correlacion_esta_presente_interno	Validación	Exige UUID de correlación válido para trazabilidad	Todos los SPs orquestadores
usp_validar_id_interno	Validación	Valida que un UUID no sea nulo ni el GUID vacío	Reutilizable por todos
usp_validar_sesion_exista_por_id_interno	Validación	Comprueba existencia de sesión en uv_sesion	usp_registrar_asistencia*, usp_cerrar_sesion, usp_radicar_solicitud*
usp_validar_estudiante_exista_por_id_interno	Validación	Comprueba existencia y estado activo del estudiante	usp_registrar_estudiante*, usp_radicar_solicitud*
usp_validar_docente_exista_por_id_interno	Validación	Comprueba existencia y estado activo del docente	usp_crear_grupo, usp_actualizar_grupo, usp_registrar_docente*
usp_validar_grupo_exista_por_id_interno	Validación	Comprueba existencia y habilitación del grupo	usp_registrar_estudiante*, usp_generar_sesiones*
usp_validar_grupo_exista_para_docente_interno	RBAC	Valida que el grupo pertenezca al docente titular	usp_crear_sesion, usp_cerrar_sesion
usp_validar_estudiante_grupo_exista_interno	Validación	Valida existencia de matrícula activa	usp_registrar_asistencia_estudiante
usp_validar_estudiante_pertenece_a_grupo_de_sesion_interno	Validación	Valida inscripción del estudiante en el grupo de la sesión	usp_registrar_asistencias_sesion, usp_radicar_solicitud*
usp_validar_registro_estudiante_en_grupo_interno	Unicidad	Valida que el estudiante no esté ya matriculado	usp_registrar_estudiante_en_grupo_interno
usp_validar_cruce_horario_estudiante_interno	Concurrencia	Detecta solapamientos horarios del estudiante	usp_registrar_estudiante_en_grupo_interno
usp_validar_cruce_horario_docente_interno	Concurrencia	Detecta solapamientos horarios del docente	usp_registrar_docente_en_grupo_interno
usp_validar_horarios_grupo_interno	Validación	Verifica franjas horarias configuradas	usp_generar_sesiones_grupo
usp_validar_fechas_periodo_academico_interno	Validación	Valida coherencia y vigencia de fechas del periodo	usp_generar_sesiones_grupo
usp_validar_unicidad_usuario_interno	Unicidad	Valida unicidad de correo y documento	usp_sincronizar_usuario_interno
usp_validar_tipo_identificacion_exista_por_id_interno	Validación	Valida existencia del tipo de identificación	usp_sincronizar_usuario_interno
usp_validar_perfil_existe_por_codigo_interno	Validación	Valida existencia de perfil por código	usp_sincronizar_*_interno
usp_validar_usuario_existe_por_id_interno	Validación	Valida existencia del usuario por ID	Reutilizable
usp_sincronizar_usuario_interno	Persistencia	Crea o actualiza registro en Usuario	usp_crear_decano, usp_crear_coordinador
usp_sincronizar_asistencia_estudiante_interno	Persistencia	MERGE idempotente en Asistencia + DetalleAsistencia	usp_registrar_asistencia*, usp_registrar_asistencias_sesion

PROCEDIMIENTO INTERNO	TIPO	PROPÓSITO	INVOCADO POR
usp_sincronizar_docente_interno	Persistencia	Crea registro Docente desde Usuario existente	DocenteRepositorySqlServerAdapter
usp_sincronizar_estudiante_interno	Persistencia	Crea registro Estudiante desde Usuario existente	Orquestadores de registro estudiantil
usp_obtener_mensaje_catalogo	Mensajería	Traduce códigos a mensajes usuario/técnico	Todos los SPs